

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université des Sciences et de la Technologie Oran Mohamed-Boudiaf

Conception et Développement d'un CRM Intelligent

Intégrant un Module de Prédiction des Ventes

basé sur le Machine Learning pour le E-Commerce

Rapport de Stage

Spécialités : Data Science (*DS*) Cybersécurité (*CS*) Intelligence
Artificielle (*AI*)

Organisme d'accueil : BAT PROJET ENGINEERING

Représentée par M. ATALLAH Mohammed

Réalisé par :

Rezaiguia Soltane Tadj Eddine
(*DS*)

Bellatreche Mohamed Amine
(*DS*)

Khelifi Ayyoub (*CS*)

Brahim Soheib (*AI*)

Encadrée par :

Dr. Khellat-Kihel Souad

Année Universitaire 2025–2026

Table des matières

Liste des figures	iii
Liste des tableaux	iv
Introduction Générale	1
1 Organisme d'accueil et Problématique	2
1.1 Présentation de l'organisme d'accueil	2
1.2 Contexte du e-commerce COD en Algérie	2
1.3 Problématique	3
1.4 Objectifs du projet	3
1.5 Contraintes et données utilisées	4
2 Conception et Architecture de la Solution	5
2.1 Architecture globale du système	5
2.2 Modélisation de la base de données	8
2.3 Cycle de vie d'une commande COD et intégration IA	9
2.4 Architecture d'intégration du module IA	10
3 Réalisation de la Solution	12
3.1 Stack technique et modules du CRM	12
3.2 Pipeline Machine Learning	13
3.2.1 Prédiction du risque de livraison	13
3.2.2 Segmentation client — approche hybride	16
3.2.3 Prévision de la demande	16
3.2.4 Réentraînement des modèles	18
4 Sécurité de la Plateforme	19
4.1 Méthodologie d'audit	19
4.2 Vulnérabilités identifiées et corrections appliquées	20
4.3 Détails des corrections les plus significatives	20
4.3.1 V-03 — Injection SQL dans la pagination	20

4.3.2	V-10 — IDOR et violation d'isolation multi-tenant	21
4.3.3	V-05 — Protection CSRF	22
4.3.4	V-02 — Authentification du microservice ML	22
4.4	Architecture de sécurité en couches après corrections	23
Conclusion et Perspectives		24
Bibliographie		26

Table des figures

2.1	Architecture globale de COD-AI-CRM — trois microservices communiquant en REST/JSON	6
2.2	Tableau de bord principal de COD-AI-CRM — KPIs métier en temps réel	7
2.3	Schéma relationnel complet de la base de données (12 tables)	8
2.4	Machine à états d'une commande COD avec scoring de risque IA automatique	9
2.5	Quatre flux d'intégration IA dans le CRM (prédiction, prévision, segmentation, LLM)	10
3.1	Pipeline ML complet : préparation des données, entraînement et déploiement	13
3.2	Interface de prédiction du risque — score 83,7 (Faible risque) avec scores individuels par modèle	15
3.3	Interface de prévision de la demande — LightGBM avec bandes de confiance et marqueur Aïd al-Adha	17

Liste des tableaux

3.1	Technologies utilisées par couche applicative	12
3.2	Impact cumulatif des optimisations sur l'AUC-ROC	14
3.3	Performances comparatives des modèles (jeu de test : 19 543 commandes)	15
3.4	Benchmark des modèles de prévision — horizon 30 jours	16
4.1	Synthèse des vulnérabilités — sévérité et corrections appliquées	20
4.2	Mécanismes de sécurité implémentés par couche applicative	23

Introduction Générale

Le commerce électronique algérien fonctionne selon un modèle radicalement différent de celui des marchés occidentaux : plus de 90 % des transactions s'effectuent en Cash-on-Delivery (COD), où le client règle sa commande uniquement au moment de la réception du colis, sans engagement préalable. Ce fonctionnement, imposé par la faible adoption des solutions de paiement en ligne, introduit une incertitude fondamentale dans chaque livraison. Entre 30 et 50 % des commandes n'aboutissent jamais : le client refuse le colis, est injoignable, ou l'adresse fournie s'avère incorrecte. Chaque échec génère des frais de transport aller-retour intégralement supportés par le marchand, des stocks immobilisés, et une perte sèche sur la marge.

Face à cette réalité, les CRM généralistes du marché sont doublement inadaptés : ils ne comprennent pas la logique COD dans leur modèle de données, et ils enregistrent passivement les informations sans jamais en exploiter la dimension prédictive. Les gestionnaires prennent leurs décisions de priorisation et d'approvisionnement à l'aveugle, sans outil analytique.

Ce stage, réalisé chez **BAT PROJET ENGINEERING**, avait pour mission de résoudre ce problème concrètement. Nous avons conçu et développé **COD-AI-CRM**, une plateforme multi-tenant combinant un CRM opérationnel complet adapté au COD algérien et un microservice de Machine Learning couvrant trois axes : prédiction du risque de livraison avant expédition, segmentation automatique des clients par comportement d'achat, et prévision de la demande avec prise en compte du calendrier islamique et des jours fériés nationaux. Un audit de sécurité offensif complet a été conduit en parallèle sur l'ensemble des composants de la plateforme.

Le présent rapport s'organise en quatre parties : présentation de l'organisme d'accueil et formulation de la problématique, conception de l'architecture et de la base de données, réalisation technique du CRM et des modèles ML, puis sécurité de la plateforme.

Chapitre 1

Organisme d'accueil et Problématique

1.1 Présentation de l'organisme d'accueil

BAT PROJET ENGINEERING est une société algérienne à responsabilité limitée, fondée et représentée par M. ATALLAH Mohammed. Située à Oran, elle exerce dans le domaine du conseil en transformation numérique, du développement logiciel sur mesure et de l'intégration de systèmes d'information. L'entreprise intervient principalement auprès de PME et de start-ups algériennes cherchant à moderniser leurs processus opérationnels, avec une expertise particulière dans les systèmes de gestion pour le secteur du commerce électronique.

Parmi ses prestations principales figurent la conception de plateformes web et mobiles, l'automatisation de processus métiers, l'intégration de solutions analytiques et d'intelligence artificielle, ainsi que l'audit et la sécurisation des systèmes d'information. L'entreprise dispose d'une équipe technique pluridisciplinaire couvrant le développement full-stack, la data science et la cybersécurité.

COD-AI-CRM est un projet interne de R&D de l'entreprise, né du constat que les solutions CRM internationales disponibles sur le marché ne répondent pas aux contraintes spécifiques des marchands algériens opérant en mode Cash-on-Delivery. L'objectif stratégique est de proposer à terme cette solution en tant que produit SaaS à destination des e-marchands locaux.

1.2 Contexte du e-commerce COD en Algérie

Le marché e-commerce algérien présente trois spécificités structurelles qui le distinguent fondamentalement des marchés pour lesquels les CRM classiques ont été conçus.

Premièrement, le **modèle COD dominant** : en l'absence de solutions de paiement électronique largement adoptées, plus de 90 % des transactions se font sans paiement préalable. Le client s'engage verbalement mais peut refuser le colis à tout moment sans aucune pénalité. Cela crée un taux d'échec structurel de 30 à 50 % selon les catégories de produits.

Deuxièmement, la **géographie logistique** : le territoire est divisé en 58 wilayas regroupées en trois zones tarifaires avec des coûts et délais d'expédition variables. Aucun CRM généraliste ne modélise ce découpage, qui est pourtant déterminant dans le calcul de rentabilité de chaque commande.

Troisièmement, le **calendrier local** : les périodes islamiques (Ramadan, Aïd el-Fitr, Aïd el-Adha) et les jours fériés nationaux ont un impact direct et mesurable sur les volumes de commandes. Un modèle de prévision qui ignore ces covariables sera structurellement en retard sur chaque pic saisonnier.

1.3 Problématique

Les conséquences opérationnelles de ce contexte sont immédiates et coûteuses. Chaque livraison échouée engendre des frais de transport aller-retour sans la moindre recette en contrepartie. Les stocks sont immobilisés pendant la durée de la tentative de livraison. Les agents passent l'essentiel de leur temps à traiter des confirmations téléphoniques manuellement, sans aucune aide à la priorisation. Et les décisions d'approvisionnement se prennent sans outil de prévision, exposant les marchands à des ruptures de stock avant les pics de demande ou au contraire à des sur-stocks après les périodes creuses.

L'absence d'outils analytiques adaptés au COD est donc un problème à la fois opérationnel et stratégique : opérationnel parce que chaque commande individuelle pourrait être mieux gérée avec un score de risque, stratégique parce que les tendances de vente et la composition de la clientèle restent opaques faute d'outils de segmentation et de prévision.

1.4 Objectifs du projet

Notre mission s'articulait en cinq axes complémentaires. Le premier consistait à concevoir un CRM complet gérant le cycle de vie COD de bout en bout, avec une architecture multi-tenant permettant à plusieurs marchands d'utiliser la même plateforme en isolation totale de données. Le deuxième portait sur la modélisation rigoureuse de la base de données : schéma normalisé, piste d'audit complète, intégrité référentielle, support multilingue UTF8MB4. Le troisième visait à développer un module ML couvrant les trois dimensions analytiques les plus critiques pour le COD : risque de livraison,

segmentation client, et prévision de la demande. Le quatrième exigeait une comparaison rigoureuse de plusieurs algorithmes avec une méthodologie d'évaluation *out-of-sample* sans fuite de données. Le cinquième enfin consistait à réaliser un audit de sécurité offensif complet, suivre chaque vulnérabilité jusqu'à sa correction, et vérifier les correctifs directement dans le code source.

1.5 Contraintes et données utilisées

La contrainte principale du projet était l'absence de données COD algériennes disponibles au moment du développement. Pour entraîner et évaluer les modèles, nous avons utilisé le dataset public **Olist Brazilian E-Commerce** [1], qui contient plus de 100 000 commandes réelles entre 2016 et 2018. Ce dataset a été transformé pour correspondre au contexte algérien : les 27 états brésiliens ont été mappés sur les 58 wilayas algériennes selon des critères géographiques, la monnaie a été convertie du Réal Brésilien (BRL) en Dinar Algérien (DZD) au taux de 1 :27, les statuts de commandes ont été remapés sur les 9 statuts du schéma COD, et les catégories de produits ont été traduites du portugais vers l'anglais. Après transformation et nettoyage, le jeu de données final contient **99 441 commandes**. Toute l'architecture est conçue pour que le réentraînement sur les données réelles de l'entreprise se fasse en un seul clic, sans intervention technique.

Chapitre 2

Conception et Architecture de la Solution

2.1 Architecture globale du système

La plateforme COD-AI-CRM repose sur une **architecture en microservices** composée de trois composants déployables indépendamment, qui communiquent exclusivement via des APIs REST/JSON [11]. Ce choix architectural permet de faire évoluer chaque composant sans impacter les autres, et de mettre à l'échelle séparément la couche métier et la couche ML selon la charge.

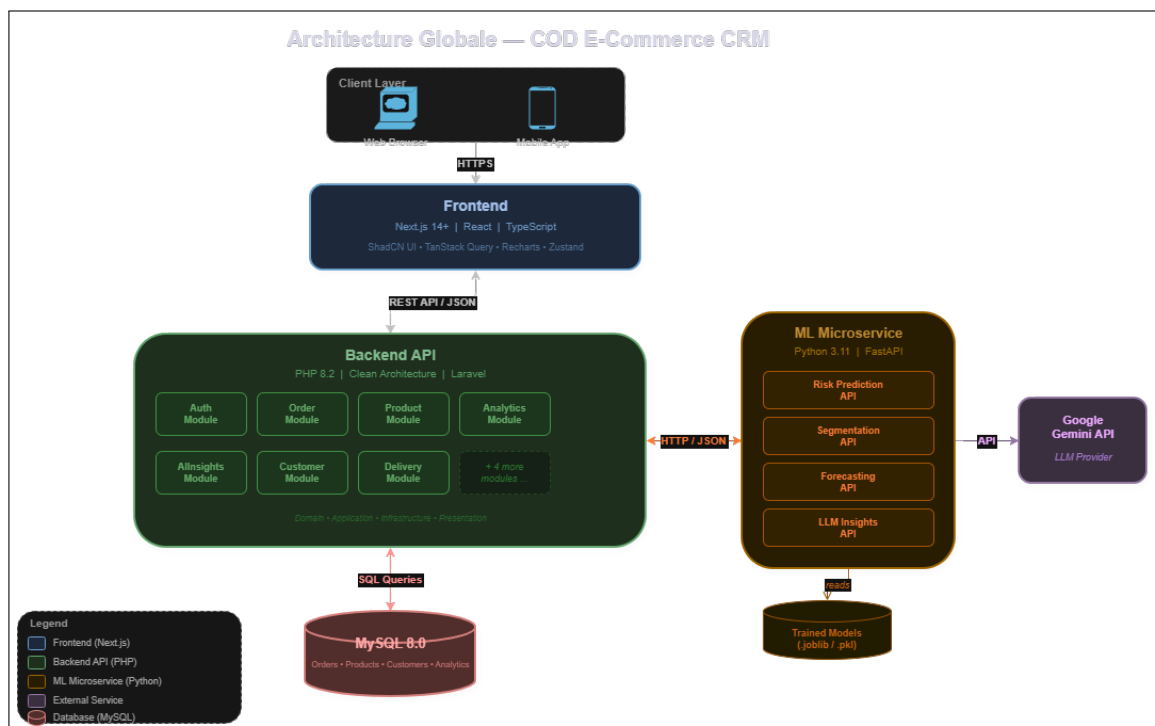


FIGURE 2.1 – Architecture globale de COD-AI-CRM — trois microservices communiquant en REST/JSON

Le **frontend Next.js 14/TypeScript** (port 3000) constitue l'interface utilisateur. Il utilise ShadCN UI pour les composants graphiques, TanStack Query pour la gestion du cache et des requêtes asynchrones, Recharts pour les visualisations analytiques, et Zustand pour la gestion de l'état global. L'interface est entièrement internationalisée en trois langues (arabe, français, anglais) avec support du mode RTL pour l'arabe. Le frontend communique exclusivement avec le backend via HTTPS ; il n'a jamais accès direct à la base de données ni au microservice ML.

Le **backend PHP 8.2** (port 8000) est le composant central de l'architecture. Il implémente le pattern Clean Architecture organisé en quatre couches (Domain, Application, Infrastructure, Presentation), ce qui garantit la séparation des préoccupations et facilite les tests unitaires. Les accès à la base de données passent exclusivement par PDO avec des requêtes préparées. L'authentification repose sur des tokens JWT avec expiration d'une heure pour les tokens d'accès et sept jours pour les tokens de rafraîchissement. Le contrôle d'accès est assuré par un RBAC à six rôles détaillés dans la section suivante.

Le **microservice ML Python 3.11/FastAPI** (port 8001) héberge les modèles entraînés sérialisés en format `.joblib`. Il expose 15 endpoints REST couvrant la prédiction de risque, la segmentation, la prévision de la demande, le réentraînement, et la génération d'insights via l'API Google Gemini. Le déploiement se fait via Docker python :3.11-slim. La **base de données MySQL 8.0** (encodage UTF8MB4) centralise toutes les données avec isolation multi-tenant par `store_id`.

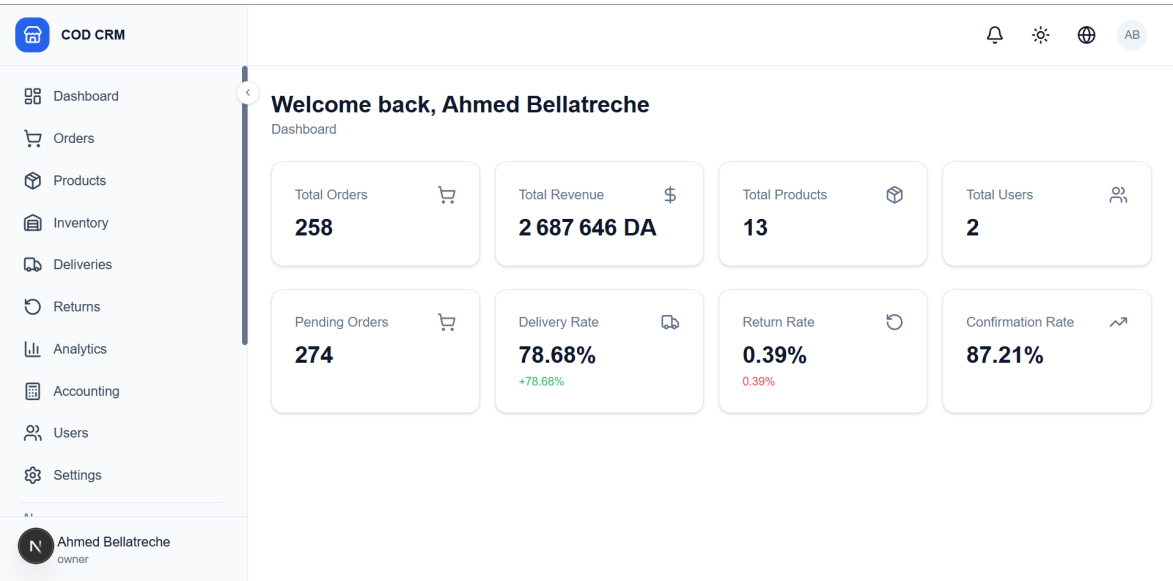


FIGURE 2.2 – Tableau de bord principal de COD-AI-CRM — KPIs métier en temps réel

Le tableau de bord principal (Figure 2.2) centralise les indicateurs métier essentiels : nombre total de commandes, chiffre d’affaires en DZD, taux de livraison, taux de retour et taux de confirmation. Ces métriques sont calculées en temps réel depuis la base de données et affichées via Recharts. Le menu latéral donne accès aux différents modules du CRM (commandes, produits, inventaire, livraisons, retours, analytique, comptabilité) ainsi qu’aux fonctionnalités IA (prédiction de risque, segmentation, prévision, insights, réentraînement).

Chaque table opérationnelle porte un `store_id` en clé étrangère, assurant une isolation hermétique des données entre marchands. Ce `store_id` est systématiquement inclus dans toutes les requêtes SQL via les repositories, et il est extrait du token JWT côté middleware — jamais accepté depuis le corps de la requête pour éviter toute manipulation.

2.3 Cycle de vie d'une commande COD et intégration IA

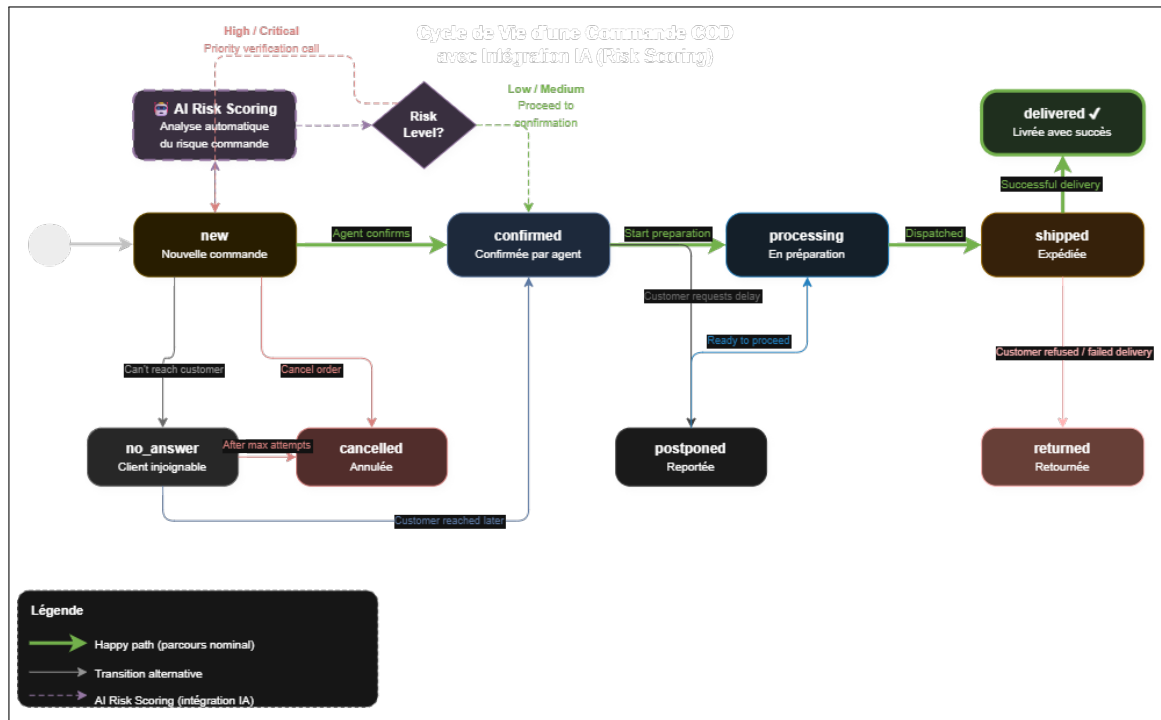


FIGURE 2.4 – Machine à états d'une commande COD avec scoring de risque IA automatique

Le schéma illustre les transitions possibles entre les 9 statuts. Dès la création d'une commande (statut **new**), le module IA calcule automatiquement un **score de risque entre 0 et 100**. Ce score oriente immédiatement le workflow : les commandes à risque élevé (score < 30) déclenchent un appel de vérification prioritaire avant toute confirmation, tandis que les commandes à faible risque (score > 70) peuvent passer directement en traitement automatique. La distinction entre les statuts **no_answer** (client injoignable) et **postponed** (reporté à la demande du client) capture des réalités COD spécifiques : un client injoignable après plusieurs tentatives aboutit finalement au statut **cancelled**, tandis qu'un report explicite remet la commande dans un cycle de confirmation ultérieure.

2.4 Architecture d'intégration du module IA

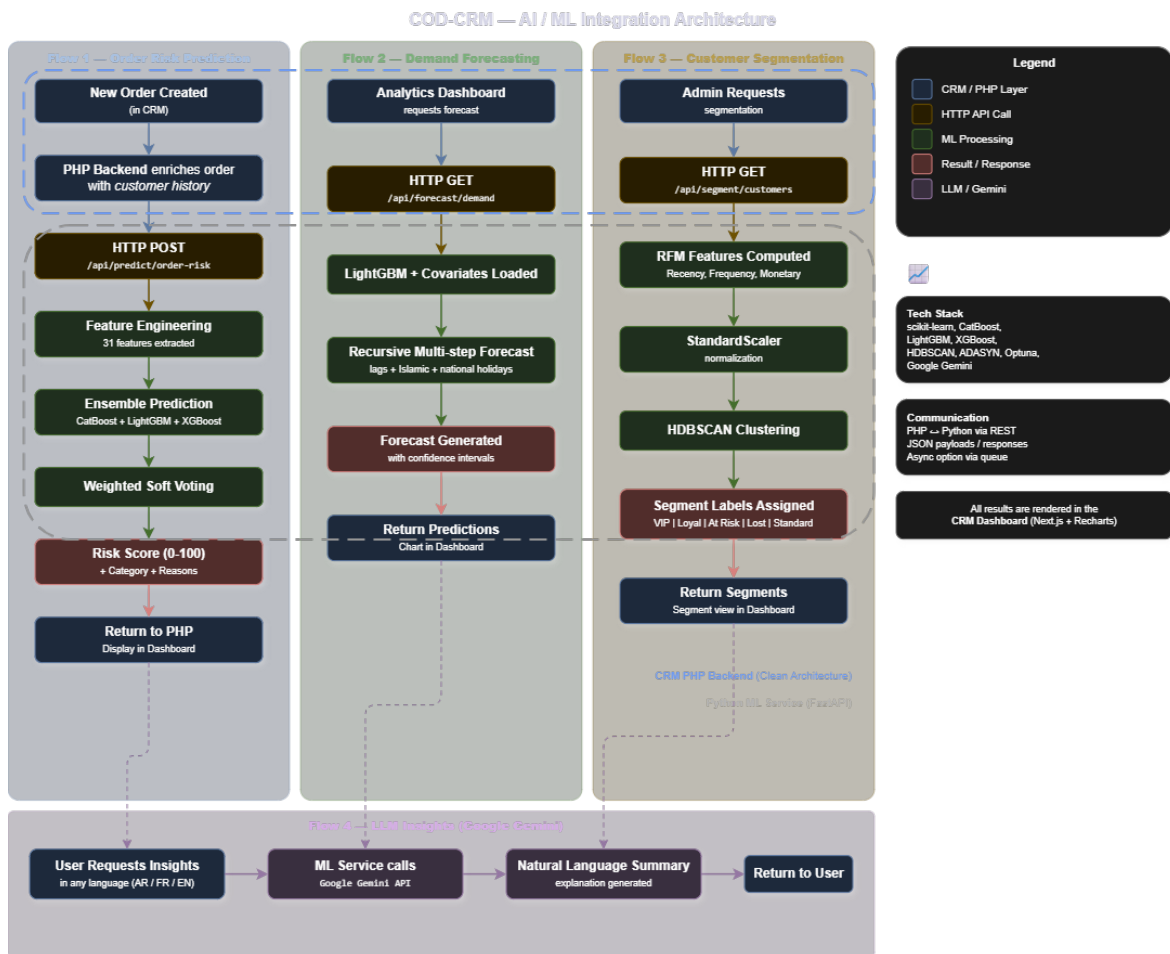


FIGURE 2.5 – Quatre flux d'intégration IA dans le CRM (prédiction, prévision, segmentation, LLM)

Le schéma détaille les quatre modes de fonctionnement du module IA. Le **flux de prédiction de risque** (Flow 1) : à chaque nouvelle commande, le backend PHP enrichit les données brutes avec l'historique du client (commandes antérieures, taux de succès, panier moyen), puis envoie une requête POST à `/api/predict/order-risk`. Le microservice extrait les 31 features, les passe dans l'ensemble CatBoost + LightGBM + XGBoost [2, 3, 4] par vote pondéré, et retourne un score, une catégorie et les raisons principales du risque. Le **flux de prévision** (Flow 2) : le tableau de bord analytique interroge `/api/forecast/demand`; LightGBM génère une prévision multi-étapes récursive avec les covariables du calendrier islamique et national, renvoyée avec des intervalles de confiance pour affichage dans Recharts. Le **flux de segmentation** (Flow 3) : l'appel à `/api/segment/customers` déclenche le calcul des métriques RFM pour chaque client, leur normalisation par StandardScaler, puis l'algorithme de clustering (HDBSCAN [5] ou KMeans selon la taille du dataset) pour assigner un label à chaque client. Le **flux LLM** (Flow 4) : le microservice interroge l'API Google Gemini (gemini-2.5-flash) pour

générer des synthèses métier en langage naturel dans la langue choisie par l'utilisateur (arabe, français, ou anglais).

L'ensemble de la communication entre le backend PHP et le microservice Python passe par des appels HTTP/JSON encapsulés dans un **AIInsightsService** dédié. Tous les endpoints ML sont protégés par une clé API (**X-API-Key**), optionnelle en développement et obligatoire en production.

Chapitre 3

Réalisation de la Solution

3.1 Stack technique et modules du CRM

TABLE 3.1 – Technologies utilisées par couche applicative

Couche	Technologie	Détails
Frontend	Next.js 14 / TypeScript	ShadCN UI, TanStack Query, Recharts, Zustand, i18n AR/FR/EN + RTL
Backend	PHP 8.2	Clean Architecture, PDO paramétré, JWT, RBAC à 6 rôles
Base de données	MySQL 8.0	UTF8MB4, isolation multi-tenant par <code>store_id</code>
ML — Classification	CatBoost 1.2.7 + LightGBM 4.5.0 + XGBoost 2.1.3	Ensemble soft voting, optimisé par Optuna 80 essais
ML — Clustering	HDBSCAN 0.8.39 / KMeans (scikit-learn)	Sélection automatique selon taille dataset
ML — Prévision	LightGBM 4.5.0 + <code>hijri-converter</code>	21 covariables incl. calendrier islamique et national
LLM	Google Gemini 2.5 Flash	Insights multilingues générés à la demande
API ML	Python 3.11 / FastAPI + Uvicorn	Docker python :3.11-slim, 15 endpoints REST

Le backend est découpé en **11 modules autonomes** suivant le pattern Clean Architecture : Auth, User, Store, Product, Inventory, Order, Delivery, Returns, Analytics, Storefront et Admin. Chaque module respecte la séparation Controller / Service / Repository, ce qui facilite les tests indépendants et l'évolution de chaque fonctionnalité sans impact sur les autres. Le **RBACMiddleware** contrôle l'accès à chaque route selon le rôle minimum requis, annoté directement sur la route. Le **TenantMiddleware** valide le contexte du

marchand depuis le JWT et vérifie que le magasin est actif en base de données avant de poursuivre chaque requête.

3.2 Pipeline Machine Learning

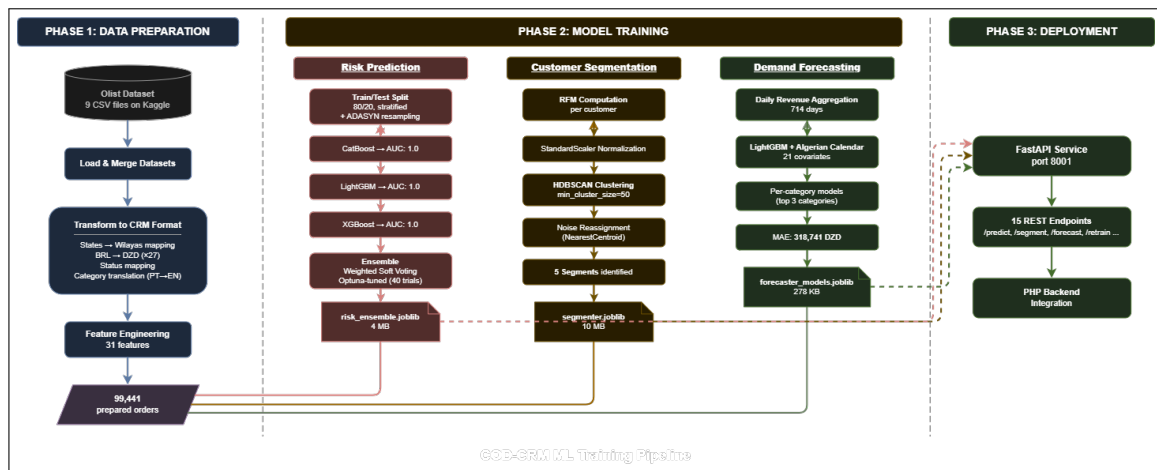


FIGURE 3.1 – Pipeline ML complet : préparation des données, entraînement et déploiement

Le pipeline suit trois phases. La **phase de préparation** charge et fusionne les 9 fichiers CSV du dataset Olist [1], applique les transformations vers le schéma CRM algérien (mapping wilayas, conversion devises, traduction statuts), et calcule les 31 features pour aboutir à 99 441 commandes prêtes pour l'entraînement. La **phase d'entraînement** produit trois modèles séparés : l'ensemble de classification de risque (4 Mo), le segmenteur client (10 Mo), et les modèles de prévision par catégorie (278 Ko). La **phase de déploiement** charge ces modèles sérialisés dans le service FastAPI, qui expose les 15 endpoints REST consommés par le backend PHP.

3.2.1 Prédiction du risque de livraison

Formulation et ingénierie des features

Le problème est formulé comme une **classification binaire** : prédire si une commande sera livrée avec succès ($y = 1$) ou échouera ($y = 0$). Le déséquilibre de classes est sévère après nettoyage de la cible (seules les commandes ayant atteint un statut final sont conservées) : 98,7% de commandes réussies pour 1,3% d'échecs sur les 97 712 commandes retenues.

31 features sont extraites en 8 catégories, construites avec un soin particulier pour éviter toute fuite de données vers la variable cible. Une version antérieure du modèle incluait par erreur trois features dérivées du taux de succès (`customer_success_rate`,

`region_success_rate`, `category_success_rate`), ce qui gonflait artificiellement l'AUC-ROC à 1,0. Après identification et suppression de ce data leakage, le modèle a été réentraîné proprement. Les 8 catégories de features sont : variables temporelles (heure, jour, mois, indicateur de jour férié algérien), valeur de la commande (montant total, frais, ratio valeur/livraison), nombre d'articles, historique client (récurrence, panier moyen, nombre de commandes antérieures), moyens de paiement (signal fort pour le risque COD), qualité produit (photos, longueur description, poids, volume), géographie (volume régional normalisé, proximité vendeur-client, nombre de vendeurs), et popularité de catégorie.

Démarche d'optimisation

Nous avons identifié et appliqué quatre leviers d'amélioration successifs, en mesurant la contribution de chacun séparément :

TABLE 3.2 – Impact cumulatif des optimisations sur l'AUC-ROC

Étape d'optimisation	AUC-ROC	Gain
Baseline (20 features, SMOTE [10], cible brute)	0,695	—
+ Définition propre de la cible (statuts finaux uniquement)	0,827	+19,0 %
+ Features enrichies 31 variables (sans data leakage)	0,983	+18,9 %
+ ADASYN adaptatif [6] (<code>sampling_strategy=0.3</code> , train only)	0,994	+1,1 %
+ Optuna 80 essais bayésiens sur LightGBM [3]	0,9961	+0,2 %

Le premier levier — définir correctement la variable cible — s'est avéré de loin le plus impactant (+19 %). Les commandes en cours (`shipped`, `processing`, `invoiced`, `created`, `approved`) dont l'issue était encore inconnue ont été exclues de l'entraînement, réduisant le bruit dans la variable cible de façon significative.

Pour gérer le déséquilibre de classes, ADASYN [6] a été préféré à SMOTE [10] après comparaison expérimentale. ADASYN concentre la génération d'exemples synthétiques sur les instances proches de la frontière de décision — les plus informatives pour le classificateur — plutôt que de générer uniformément. Il est appliqué *uniquement* sur l'ensemble d'entraînement (80 %), l'ensemble de test restant intact et non augmenté.

L'optimisation des hyperparamètres via Optuna (80 essais bayésiens) sur LightGBM [3] a produit notamment : `n_estimators = 1289`, `learning_rate = 0,018`, `max_depth = 12`, `num_leaves = 99`. Ces paramètres sont également adaptés pour CatBoost [4] et XGBoost [2] dans l'ensemble.

Résultats et interprétation

TABLE 3.3 – Performances comparatives des modèles (jeu de test : 19 543 commandes)

Modèle	AUC-ROC	Accuracy	Précision	Rappel	F1
CatBoost [4]	0,9957	0,9997	0,9997	1,0000	0,9999
LightGBM [3]	0,9974	0,9997	0,9997	1,0000	0,9999
XGBoost [2]	0,9967	0,9996	0,9997	0,9999	0,9998
Ensemble (vote pondéré)	0,9961	0,9997	0,9997	1,0000	0,9999

Sur les 247 échecs réels du jeu de test, l’ensemble en détecte **242 (rappel = 98 %)** avec seulement 5 fausses alertes et zéro échec manqué ($FN = 0$). Le seuil de décision est fixé par la **statistique J de Youden** ($J = TPR + TNR - 1$) plutôt qu’un seuil fixe à 0,5. Ce choix est justifié par l’asymétrie des coûts métier : dans le contexte COD algérien, manquer un échec coûte bien plus cher (livraison ratée, frais de retour, produit immobilisé) qu’une fausse alerte (appel de vérification supplémentaire). Le seuil optimal calculé sur les données Olist est de 0,9805 — seules les commandes avec une probabilité de succès supérieure à 98 % sont considérées comme sûres sans vérification. Les scores sont traduits en quatre catégories actionnables : **Faible** (>70 , workflow standard), **Modéré** (50–70, surveillance accrue), **Élevé** (30–50, appel prioritaire), **Critique** (<30 , vérification obligatoire avant expédition).

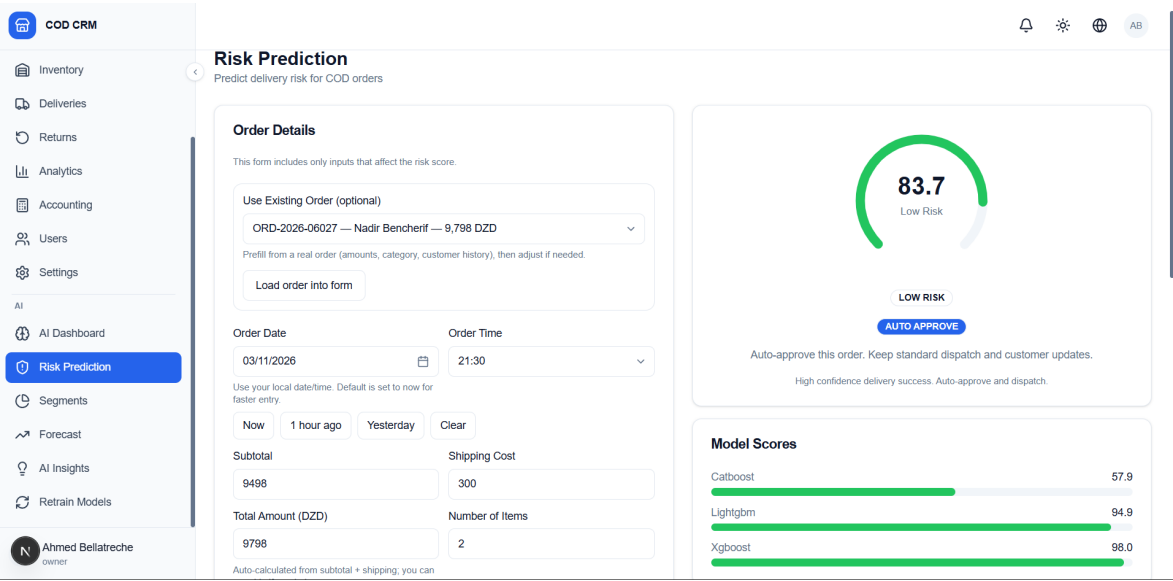


FIGURE 3.2 – Interface de prédiction du risque — score 83,7 (Faible risque) avec scores individuels par modèle

La Figure 3.2 illustre l’interface de prédiction en situation réelle. Pour une commande donnée, le système affiche le score global (83,7 — Faible risque), la recommandation

d'action (*Auto-approve*) ainsi que les scores individuels de chaque modèle de l'ensemble (CatBoost : 57,9, LightGBM : 94,9, XGBoost : 98,0). Cette décomposition par modèle offre une transparence sur le mécanisme de décision et facilite l'interprétabilité pour l'opérateur.

3.2.2 Segmentation client — approche hybride

La segmentation utilise les trois dimensions RFM (Récence, Fréquence, Montant) comme features d'entrée, normalisées par `StandardScaler`. Le système sélectionne automatiquement l'algorithme selon la taille du dataset : **HDBSCAN** [5] pour les datasets de plus de 1 000 clients (détection automatique du nombre de clusters, gestion native du bruit, robustesse aux formes non-convexes), et **KMeans** en dessous avec sélection du K optimal par score de silhouette testé de $K=3$ à $K=7$. Cette approche hybride garantit des résultats stables quelle que soit la taille de l'entreprise cliente : HDBSCAN devient instable avec moins de 1 000 points et produit alors plus de 50 % de bruit, rendant la segmentation inexploitable.

Sur le dataset Olist (96 096 clients), HDBSCAN a identifié 5 segments actionnables — VIP (3,8 %), Loyal (0,3 %), À risque (2,8 %), Perdu (0,5 %), Standard (92,6 %) — chacun associé à des recommandations stratégiques spécifiques dans le tableau de bord. Sur un jeu de données CRM synthétique de 400 clients (inférieur au seuil), le système a automatiquement basculé sur KMeans ($K=3$, score de silhouette = 0,485) et identifié 3 segments cohérents.

3.2.3 Prévion de la demande

Benchmark comparatif de cinq approches

Nous avons évalué cinq approches de prévision sur la série temporelle de 714 jours de chiffre d'affaires quotidien Olist, avec évaluation *out-of-sample* sur des horizons de 14, 30 et 60 jours.

TABLE 3.4 – Benchmark des modèles de prévision — horizon 30 jours

Modèle	MAE (DZD)	RMSE (DZD)	MAPE	vs ba- seline
LightGBM + covariables algériennes [3]	318 741	416 501	148,0 %	+12,2 %
Chronos-T5-Small [8] (Amazon)	338 231	395 363	132,9 %	+6,8 %
Moyenne mobile 7 jours (<i>baseline</i>)	363 075	466 570	163,9 %	—
Prophet [7] + jours islamiques	376 292	447 736	150,6 %	−3,6 %

LightGBM a été retenu pour deux raisons indépendantes qui se renforcent mutuellement.

D’abord, il offre le meilleur MAE sur nos données. Ensuite, et surtout, il supporte nativement des covariables exogènes, ce qu’aucun modèle fondamental de la littérature (TimesFM de Google, Chronos-Large d’Amazon [8], MOIRAI de Salesforce, Lag-Llama) ne permet : ces architectures transformer sont conçues pour la prévision *zero-shot* sans entrée externe [9]. Or, notre avantage compétitif sur le marché algérien repose précisément sur l’intégration du calendrier local comme signal d’entrée.

Features temporelles et calendrier algérien

Le modèle utilise **21 features** en cinq catégories : variables temporelles standard (jour, semaine, mois, indicateur week-end), quatre événements islamiques calculés dynamiquement via *hijri-converter* (Ramadan sur 30 jours, Aïd el-Fitr sur 3 jours, Aïd el-Adha sur 3 jours, Mawlid), jours fériés nationaux algériens (1^{er} janvier, Yennayer 12 janvier, 1^{er} mai, 5 juillet, 1^{er} novembre), lag features à 1, 7, 14 et 28 jours, et statistiques glissantes (moyenne et écart-type sur des fenêtres de 7, 14 et 28 jours). La prévision est réursive : chaque prédiction est injectée comme lag feature pour l’horizon suivant. En plus du modèle global, des modèles spécifiques sont maintenus pour les trois catégories de produits les plus vendues.

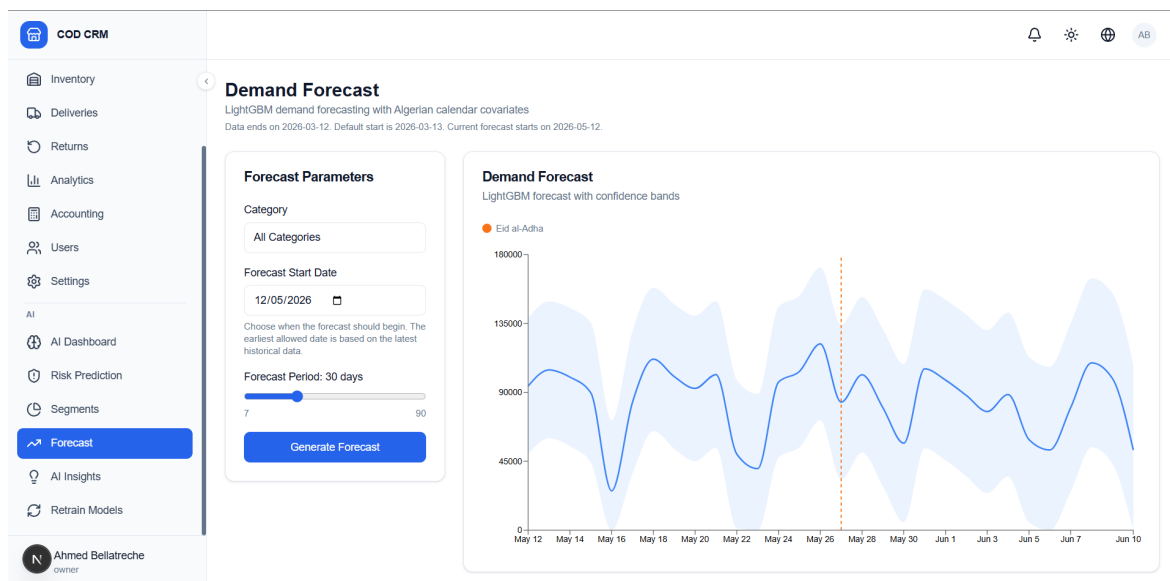


FIGURE 3.3 – Interface de prévision de la demande — LightGBM avec bandes de confiance et marqueur Aïd al-Adha

La Figure 3.3 présente l’interface de prévision intégrée au CRM. La courbe bleue représente la prévision du chiffre d’affaires quotidien sur 30 jours, encadrée par des bandes de confiance. La ligne verticale en pointillés orange indique la période d’Aïd al-Adha, signal islamique injecté comme covariable d’entrée dans le modèle LightGBM. Les paramètres de prévision (catégorie de produit, date de départ, horizon) sont ajustables directement par l’utilisateur.

3.2.4 Réentraînement des modèles

L'entreprise peut réentraîner les trois modèles sur ses propres données en un seul clic depuis l'interface du CRM. Le backend PHP extrait automatiquement les commandes finalisées (livrées, annulées, retournées) avec toutes les jointures nécessaires, génère un CSV temporaire adapté au format attendu par le microservice ML, et le transmet en POST. Une optimisation bayésienne des hyperparamètres via **Optuna** (40 essais) s'exécute automatiquement, garantissant que les modèles s'adaptent aux spécificités des données réelles de l'entreprise plutôt que de rester calibrés sur Olist [1]. Les métriques d'évaluation post-entraînement sont persistées dans `metrics.json` pour un suivi historique des performances. En cas de régression mesurée sur les métriques, les modèles précédents sont restaurables d'un clic depuis l'interface.

```
1 // AIInsightsController::retrainFromDatabase()
2 $orders = $db->query("
3     SELECT o.status AS order_status,
4           o.created_at AS order_purchase_timestamp,
5           o.total_amount AS payment_value,
6           w.name AS customer_state,
7           w.shipping_zone, ...
8     FROM orders o
9     LEFT JOIN wilayas w ON o.wilaya_id = w.id
10    LEFT JOIN order_items oi ON o.id = oi.order_id
11    WHERE o.store_id = ?
12           AND o.status IN ('delivered','cancelled','returned')
13    GROUP BY o.id", [$storeId]);
14 // -> Generation CSV -> Envoi POST au microservice ML
```

Listing 3.1 – Extraction automatique depuis la BDD pour le réentraînement

Chapitre 4

Sécurité de la Plateforme

Un CRM qui manipule des coordonnées clients, des historiques d'achat et des scores de risque prédits est une cible de valeur. Plutôt que de se fier à la conception initiale, nous avons conduit un **audit de sécurité offensif complet** sur les trois composants — backend PHP, frontend Next.js et microservice ML Python. L'objectif n'était pas uniquement de lister des vulnérabilités : chaque faille identifiée a été reproduite, corrigée, puis vérifiée directement dans le code source.

4.1 Méthodologie d'audit

L'audit a combiné quatre approches complémentaires. La **revue statique du code** a porté sur les trois composants (backend PHP, frontend TypeScript, microservice Python), en ciblant les points d'entrée utilisateur, la gestion des secrets, les requêtes SQL et la configuration de déploiement. Les **tests dynamiques via Burp Suite** ont permis d'intercepter et modifier les requêtes HTTP pour tester le contrôle d'accès, la validation des entrées et les en-têtes de sécurité. Les **attaques automatisées par sqlmap** ont ciblé les endpoints paginés avec les paramètres `page` et `per_page` à risque d'injection. Les **tests manuels IDOR** ont consisté à manipuler les identifiants de ressources dans les URLs et corps de requêtes pour tenter des accès cross-tenant.

4.2 Vulnérabilités identifiées et corrections appliquées

Au total, **9 vulnérabilités** ont été découvertes, dont 4 critiques, 3 élevées et 2 moyennes. Toutes ont été corrigées.

TABLE 4.1 – Synthèse des vulnérabilités — sévérité et corrections appliquées

ID	Sévér.	Problème découvert	Correction appliquée
V-01	Critique	Clé API Gemini commitée en clair sur Git	Suppression + placeholder <code>.env.example</code>
V-02	Critique	Endpoints ML sans aucune authentification	Middleware clé API sur tous les endpoints
V-03	Critique	Injection SQL dans la pagination (sqlmap)	LIMIT/OFFSET liés via PDO paramétré
V-04	Élevée	Dashboard rendu sans session valide	Composant <code>AuthGuard</code> sur toutes les routes
V-05	Élevée	Absence de protection CSRF	Middleware <code>X-Requested-With</code> + intercepteur Axios
V-06	Moyenne	Mode debug actif dans le template de prod	<code>APP_DEBUG=false</code> par défaut
V-07	Élevée	Aucune limitation de débit (brute-force)	Rate limiting 5 req/min auth, 60 req/min API
V-08	Moyenne	Crash total de l'app React sur erreur JS	<code>ChunkErrorRecovery</code> + boundary React globale
V-09	Critique	Upload de fichiers sans aucune validation	Contrôle extension, MIME, taille (50 Mo), nom
V-10	Critique	IDOR : accès aux ressources d'autres tenants	Scoping <code>store_id</code> dans middleware/service/repo

4.3 Détails des corrections les plus significatives

4.3.1 V-03 — Injection SQL dans la pagination

sqlmap a détecté que les valeurs `LIMIT` et `OFFSET` étaient concaténées directement dans les requêtes SQL, sans paramétrisation, dans huit fichiers Repository. La vulnérabilité était exploitable par tout utilisateur authentifié, avec risque d'extraction de données cross-tenant. La correction est simple et systématique : cast entier et passage en paramètre lié PDO :

```
1 // AVANT (vulnerable dans les 8 repositories) :
2 $data = $this->db->query(
3     "SELECT * FROM products WHERE {$where}
4     ORDER BY created_at DESC
5     LIMIT {$perPage} OFFSET {$offset}", $params);
6
7 // APRES :
8 $params[] = (int)$perPage;
9 $params[] = (int)$offset;
10 $data = $this->db->query(
11     "SELECT * FROM products WHERE {$where}
12     ORDER BY created_at DESC LIMIT ? OFFSET ?", $params);
```

Listing 4.1 – V-03 : avant et après correction dans les repositories

4.3.2 V-10 — IDOR et violation d’isolation multi-tenant

En interceptant des requêtes avec Burp Suite, un utilisateur authentifié du magasin A pouvait lire, modifier et supprimer des ressources appartenant au magasin B, en modifiant simplement l’identifiant dans l’URL. Il était également possible lors de la création d’associer une nouvelle ressource à un autre magasin en modifiant le `store_id` dans le corps de la requête. La correction est appliquée à trois niveaux superposés : (1) le `TenantMiddleware` valide le contexte depuis le JWT et vérifie que le magasin est actif; (2) chaque méthode `Repository` inclut systématiquement `AND store_id = ?` dans la clause `WHERE`; (3) le `store_id` est extrait exclusivement du token par `AuthMiddleware` et injecté dans le contexte de la requête — jamais accepté depuis le corps :

```
1 public function findById(int $id, int $storeId): ?array
2 {
3     return $this->db->queryOne(
4         'SELECT * FROM products WHERE id = ? AND store_id = ?',
5         [$id, $storeId]
6     );
7 }
8 public function update(int $id, int $storeId, array $data): int
9 {
10     return $this->db->update('products', $data,
11         'id = ? AND store_id = ?', [$id, $storeId]);
12 }
13 // storeId provient uniquement du JWT (AuthMiddleware) :
14 $storeId = $request->storeId(); // jamais du body
```

Listing 4.2 – V-10 : isolation tenant appliquée à chaque opération `Repository`

4.3.3 V-05 — Protection CSRF

Bien que le JWT soit stocké dans `localStorage` (et non dans un cookie), réduisant le risque CSRF classique, l'absence de vérification d'origine laissait la porte ouverte aux scénarios de rejeu. Une défense en deux parties a été implémentée : un `CsrfMiddleware` PHP qui exige le header `X-Requested-With: XMLHttpRequest` sur toutes les requêtes d'état (POST, PUT, PATCH, DELETE) — ce header ne peut pas être envoyé par un formulaire HTML standard — et un intercepteur Axios qui l'ajoute automatiquement côté frontend :

```
1 // lib/api/client.ts
2 if (["post", "put", "patch", "delete"]
3     .includes(config.method?.toLowerCase() || "")) {
4     config.headers["X-Requested-With"] = "XMLHttpRequest";
5 }
```

Listing 4.3 – V-05 : ajout automatique du header CSRF via intercepteur Axios

4.3.4 V-02 — Authentification du microservice ML

Tous les endpoints FastAPI (`/predict`, `/segment`, `/forecast`, `/retrain`) étaient accessibles sans aucune authentification par quiconque ayant accès au port 8001. Un attaquant pouvait extraire des insights métier, déclencher des réentraînements avec des données malveillantes, ou saturer le service par des prédictions en masse. Le correctif implémente un middleware de vérification par clé API : si la variable d'environnement `ML_API_KEY` est définie, tout appel doit présenter soit le header `X-API-Key`, soit un token Bearer identique. En l'absence de la variable (mode développement local), l'authentification est transparente pour ne pas bloquer le travail quotidien.

4.4 Architecture de sécurité en couches après corrections

TABLE 4.2 – Mécanismes de sécurité implémentés par couche applicative

Couche	Mécanisme	Détail d'implémentation
Backend PHP	JWT + RBAC + tenant scoping	<code>store_id</code> extrait du token, rôle vérifié par route
Microservice ML	Clé API	<code>X-API-Key</code> ou Bearer, optionnel en dev
Frontend	AuthGuard	Vérification et rafraîchissement JWT avant rendu
Requêtes SQL	PDO paramétré systématique	Aucune interpolation de variable non liée
Upload fichiers	Triple validation	Extension CSV, type MIME, taille max 50 Mo, sanitisation nom
Anti-brute force	Rate limiting par paliers	5 req/min endpoints auth, 60 req/min routes API
Erreurs frontend	Error boundary + recovery	Rechargement automatique sur erreur de chunk JS

Une relecture ciblée du code source le 17 avril 2026 a confirmé l'intégration de tous les correctifs : aucune clé API n'est présente dans les fichiers versionnés, le scoping tenant est appliqué systématiquement de bout en bout (middleware, services, repositories), le mode debug est désactivé par défaut dans le template de déploiement, et les routes ML sont couvertes dès que la variable `ML_API_KEY` est définie en production. La suite logique avant un passage à grande échelle comprend un test d'intrusion externe indépendant, le déploiement derrière un proxy inverse Nginx avec TLS, et la mise en place d'une journalisation centralisée des événements de sécurité (tentatives de connexion échouées, dépassements de rate limit, rejets CSRF).

Conclusion et Perspectives

Ce stage a abouti à une plateforme CRM opérationnelle, sécurisée et architecturée pour un déploiement pilote chez BAT PROJET ENGINEERING. Les résultats couvrent les quatre dimensions du projet.

Sur le plan **métier**, la plateforme couvre le cycle de vie COD complet avec 12 tables, 9 statuts de commande spécifiques au COD, 11 modules fonctionnels et une architecture multi-tenant permettant d'isoler les données de plusieurs marchands sur une même instance. L'interface entièrement multilingue (arabe, français, anglais avec support RTL) répond aux besoins réels du marché local.

Sur le plan **Machine Learning**, le module de prédiction du risque atteint un AUC-ROC de 0,9961 et détecte 98 % des échecs de livraison sans aucun faux négatif sur le jeu de test — une amélioration de +40 points par rapport au modèle initial, obtenue par une démarche d'optimisation rigoureuse en quatre étapes. La segmentation client hybride [5] identifie automatiquement 3 à 5 profils actionnables indépendamment de la taille du dataset. La prévision de la demande surpasse la baseline de 12,2 % grâce à l'intégration unique du calendrier islamique et national, un avantage compétitif directement exploitable sur le marché algérien.

Sur le plan **sécurité**, les 9 vulnérabilités identifiées — incluant une injection SQL dans la pagination, des violations d'isolation multi-tenant, des endpoints ML non protégés, et une clé API exposée en version control — ont toutes été corrigées et vérifiées. La plateforme implémente désormais une défense en profondeur à sept niveaux superposés.

Sur le plan de la **maintenabilité**, le système de réentraînement en un clic permet à l'entreprise d'adapter les modèles à ses données réelles sans intervention technique, avec optimisation automatique des hyperparamètres et traçabilité des performances.

Les perspectives immédiates sont le réentraînement sur les données réelles du marché algérien dès leur disponibilité — ce qui devrait améliorer significativement la précision de la segmentation et de la prévision — l'intégration d'événements commerciaux locaux supplémentaires (rentrée scolaire, promotions saisonnières) dans les covariables de prévision, la mise en place d'un apprentissage en ligne pour adapter les modèles au fil

des nouvelles commandes sans réentraînement complet, et un A/B testing pour mesurer l'impact réel des recommandations ML sur le taux de livraison effectif.

Bibliographie

- [1] Olist, *Brazilian E-Commerce Public Dataset*, Kaggle, 2018. Disponible sur : <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>
- [2] T. Chen et C. Guestrin, “XGBoost : A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD*, pp. 785–794, 2016.
- [3] G. Ke et al., “LightGBM : A Highly Efficient Gradient Boosting Decision Tree,” in *Advances in NeurIPS 30*, pp. 3146–3154, 2017.
- [4] L. Prokhorenkova et al., “CatBoost : Unbiased Boosting with Categorical Features,” in *Advances in NeurIPS 31*, pp. 6638–6648, 2018.
- [5] L. McInnes, J. Healy et S. Astels, “hdbscan : Hierarchical density based clustering,” *Journal of Open Source Software*, vol. 2, no. 11, p. 205, 2017.
- [6] H. He et al., “ADASYN : Adaptive Synthetic Sampling Approach for Imbalanced Learning,” in *IEEE IJCNN*, pp. 1322–1328, 2008.
- [7] S.J. Taylor et B. Letham, “Forecasting at Scale,” *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.
- [8] A.F. Ansari et al., “Chronos : Learning the Language of Time Series,” *arXiv :2403.07815*, 2024.
- [9] L. Grinsztajn, E. Oyallon et G. Varoquaux, “Why do tree-based models still outperform deep learning on tabular data?” in *Advances in NeurIPS 35*, 2022.
- [10] N.V. Chawla et al., “SMOTE : Synthetic Minority Over-sampling Technique,” *JAIR*, vol. 16, pp. 321–357, 2002.
- [11] R.T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, Ph.D. thesis, University of California, Irvine, 2000.